

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна.

Студент гр. 4345

Беденков А.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы:

Изучить и реализовать метод нахождения редакционного расстояния.

Задание:

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую. Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` -> `stal`.
- Затем необходимо заменить два последних символа: `stal` -> `stie`.
- Потом нужно добавить символ в конец строки: `stie` -> `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. $(S, 1 \leq |S| \leq 2550)$.

Вторая строка входных данных содержит строку из строчных латинских букв. $(T, 1 \leq |T| \leq 2550)$.

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Тестовые данные

№ Теста	Входные данные	Выходные данные
1	pedestal stien	7

Вар. 11. Среди всех префиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти префиксы. Среди всех суффиксов первой строки найти те, редакционное расстояние от которых до второй строки минимально, вывести это расстояние и эти суффиксы. Асимптотическая сложность решения задачи должна быть не больше, чем при обычном вычислении редакционного расстояния.

Выполнение работы

Для решения поставленной задачи используется **алгоритм Вагнера — Фишера**, основанный на методе динамического программирования. Заводится матрица **dp** размером $(N+1) \times (M+1)$, где N — длина первой строки A , а M — длина второй строки B .

Ячейка матрицы **dp[i][j]** хранит минимальную стоимость преобразования префикса строки A длины i в префикс строки B длины j . Заполнение таблицы происходит по следующей рекуррентной формуле:

$$dp[i][j] = \min(\begin{aligned} & dp[i-1][j] + 1, && // \text{удаление} \\ & dp[i][j-1] + 1, && // \text{вставка} \\ & dp[i-1][j-1] + c && // \text{замена или совпадение} \end{aligned})$$

Где $c = 0$, если текущие символы совпадают ($A[i-1] == B[j-1]$), и $c = 1$ в случае их несовпадения (требуется замена). В формуле учтены три базовые операции:

- $dp[i-1][j] + 1$ — удаление символа;
- $dp[i][j-1] + 1$ — вставка символа;
- $dp[i-1][j-1] + c$ — замена символа или совпадение.

Особенностью разработанной реализации является пошаговое логирование процесса заполнения матрицы. На каждом этапе программа выводит в консоль информацию о совершаемой операции (вставка, удаление, замена или отсутствие изменений) с указанием индексов ячеек, из которых был совершен оптимальный переход. Это позволяет наглядно отследить процесс нахождения редакционного расстояния.

После полного прохода нас интересуют значения последнего столбца матрицы — **dp[i][M]** для всех i от 0 до N . Они показывают стоимость превращения каждого возможного префикса $A[:i]$ в полную строку B .

Поиск минимальных суффиксов

Суффикс строки A длины L идентичен перевернутому префиксу длины L перевернутой строки A . Для нахождения суффиксов с минимальным редакционным расстоянием до строки B применяется тот же алгоритм, но на вход подаются реверсированные строки A и B (в коде это реализовано как $s[::-1]$ и $t[::-1]$).

Такой подход требует двукратного вычисления матрицы расстояний (для прямого и обратного прохода). Общая асимптотическая временная сложность составляет $O(N \cdot M)$, а пространственная — $O(N \cdot M)$, что соответствует стандартной сложности алгоритма вычисления расстояния Левенштейна.

Результаты тестирования

В Таблице 1 представлены результаты прогона программы на различных наборах входных данных.

Входные данные	Выходные данные	Комментарии
python thon	Минимальное расстояние (префиксы): 2 'python' Минимальное расстояние (суффиксы): 0 'thon'	Строка B является точным суффиксом строки A . Обратный проход алгоритма корректно находит идеальное совпадение с расстоянием 0. Для префиксов оптимальным решением является удаление символов 'p' и 'y'.

micro macro	Минимальное расстояние (префиксы): 1 'micro' Минимальное расстояние (суффиксы): 1 'micro'	Строки имеют одинаковую длину и отличаются ровно на один символ в середине. Алгоритм фиксирует 1 операцию (замена 'i' на 'a') как для префиксов, так и для суффиксов.
alfa alfabet	Минимальное расстояние (префиксы): 3 'alfa' Минимальное расстояние (суффиксы): 3 'alfa'	Строка А полностью входит в начало строки В. Для совпадения требуется добавить недостающие символы ('b', 'e', 't'). Программа фиксирует минимальное расстояние 3 (три операции вставки).
x x	Минимальное расстояние (префиксы): 0 'x' Минимальное расстояние (суффиксы): 0 'x'	Вырожденный случай: строки состоят из одного символа и полностью идентичны. Расстояние равно 0, модификации не требуются.

abcde bcd	Минимальное расстояние (префиксы): 1 Префиксы: 'abcd' Минимальное расстояние (суффиксы): 1 Суффиксы: 'bcde'	Строка В является подстрокой А (находится в центре). Лучший префикс 'abcd' требует 1 удаления ('a'). Лучший суффикс 'bcde' требует 1 удаления ('e').
----------------------------	--	--

Вывод

В ходе выполнения работы была изучена и реализована алгоритмическая модель вычисления редакционного расстояния на основе метода динамического программирования. Был успешно развернут функционал точечного поиска префиксов и суффиксов первой строки, обладающих наименьшим расстоянием до целевой строки В.

Для точного анализа суффиксов применен метод инверсии последовательностей, позволивший эффективно свести задачу к уже решенной схеме префиксов. Благодаря встроенному пошаговому логированию переходов, алгоритм стал прозрачным для верификации и детального разбора цепочек операций. Временная и пространственная сложность разработанного программного решения удержана в оптимальных пределах $O(N \cdot M)$.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

lb3.py

```
def get_distances(s, t):
    N, M = len(s), len(t)
    dp = [[0] * (M + 1) for _ in range(N + 1)]

    for j in range(1, M + 1):
        dp[0][j] = j
        print(f"Ячейка 0,{j}: пустая строка -> '{t[:j]}' (+
вставка)")

    for i in range(1, N + 1):
        dp[i][0] = i
        print(f"Ячейка {i},0: '{s[:i]}' -> пустая строка (+
удаление)")

    for j in range(1, M + 1):
        c = s[i-1] != t[j-1]
        dp[i][j] = min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-
1][j-1] + c)

        if not c:
            op = f"== '{t[j-1]}' -> без изменений (из {i-
1},{j-1})"

        elif dp[i][j] == dp[i-1][j-1] + 1:
            op = f"!= '{t[j-1]}' -> замена (из {i-1},{j-1})"
        elif dp[i][j] == dp[i-1][j] + 1:
            op = f"!= '{t[j-1]}' -> удаление (из {i-1},{j})"
        else:
            op = f"!= '{t[j-1]}' -> вставка (из {i},{j-1})"
        print(f"Ячейка {i},{j}: '{s[i-1]}' {op}")

    print()
    for row in dp: print(row)
    return [dp[i][M] for i in range(N + 1)]

def solve():
```

```

s, t = input().strip(), input().strip()

print("--- ПРЯМОЙ ПРОХОД (ПРЕФИКСЫ) ---")
p_dists = get_distances(s, t)

print("\n--- ОБРАТНЫЙ ПРОХОД (СУФФИКСЫ) ---")
s_dists = get_distances(s[::-1], t[::-1])

min_p, min_s = min(p_dists), min(s_dists)

print("\nИтоговое расстояние:", p_dists[-1])
print("\nМинимальное расстояние (префиксы):", min_p)
for i, d in enumerate(p_dists):
    if d == min_p: print(f"  {s[:i]} or '(пустая строка)'}'")

print("Минимальное расстояние (суффиксы):", min_s)
for i, d in enumerate(s_dists):
    if d == min_s: print(f"  {s[len(s)-i:] or '(пустая строка)'}'")

if __name__ == '__main__':
    solve()

```

.